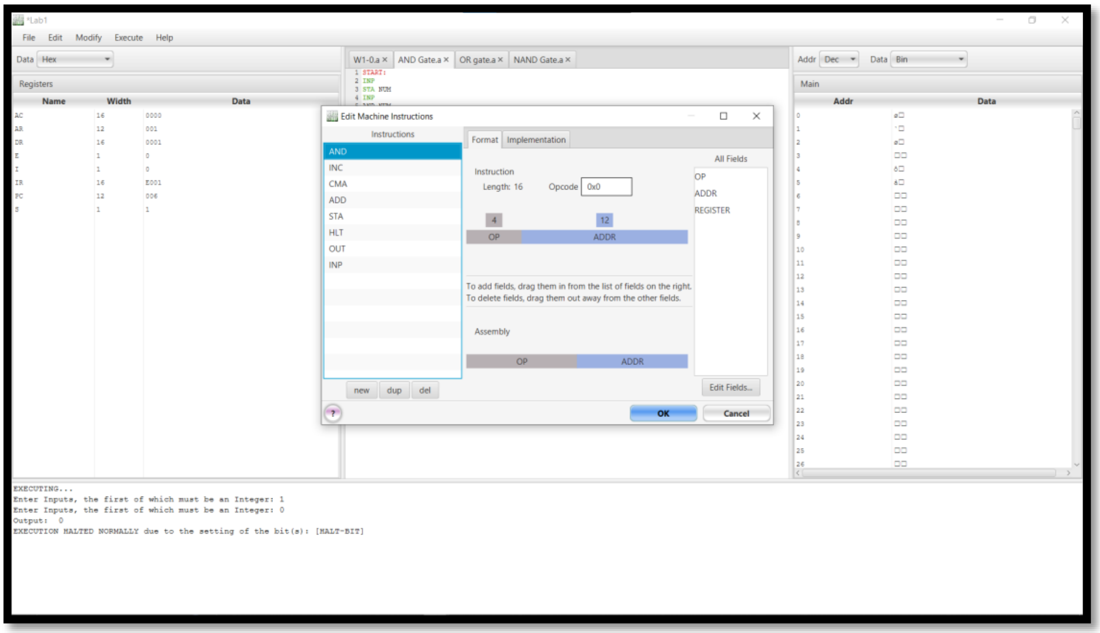
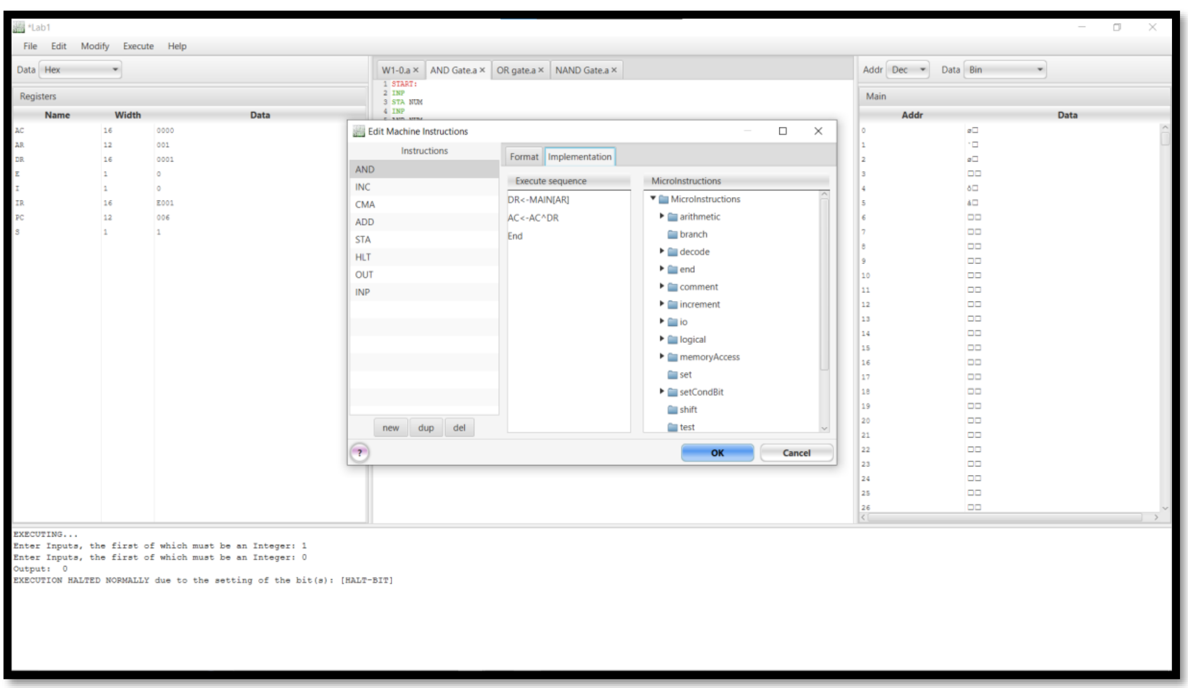
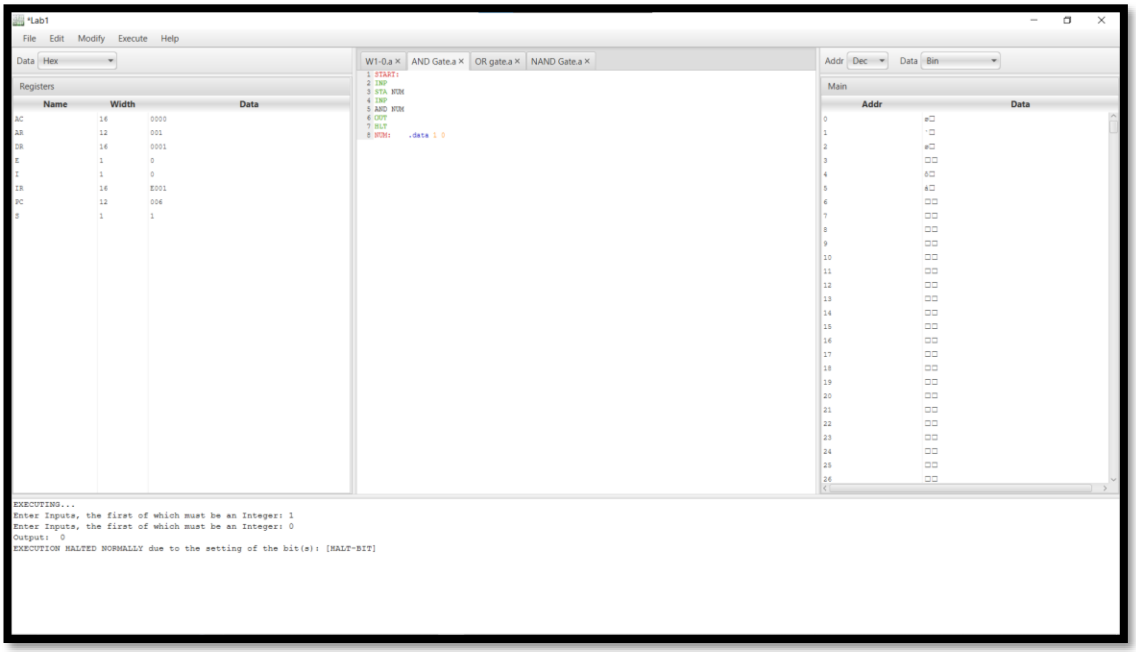
Task # 1:

AND Gate:



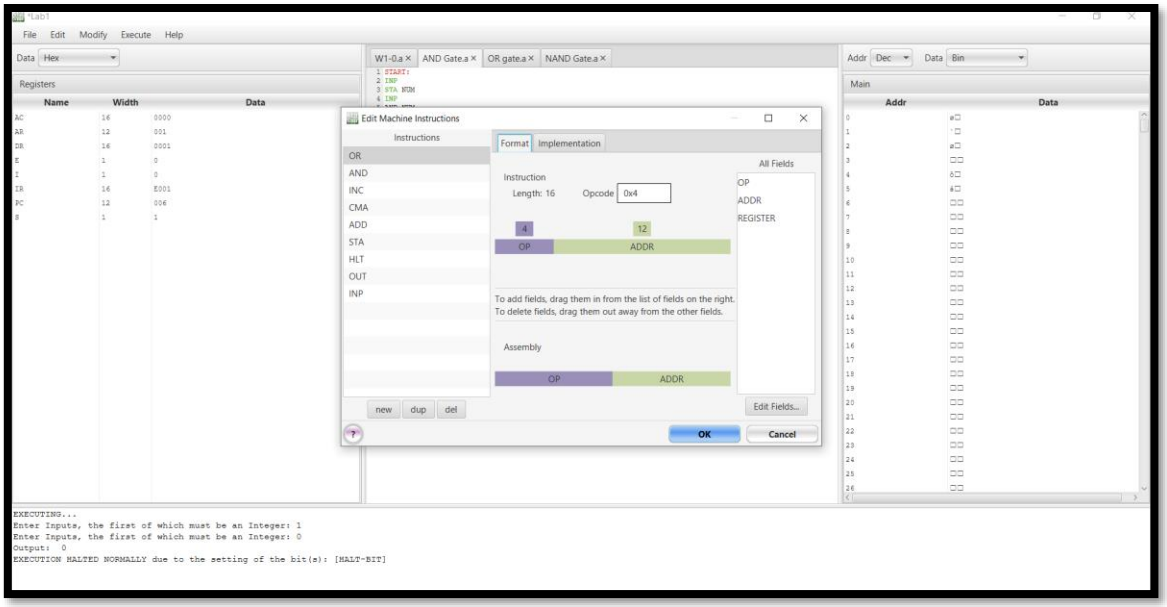


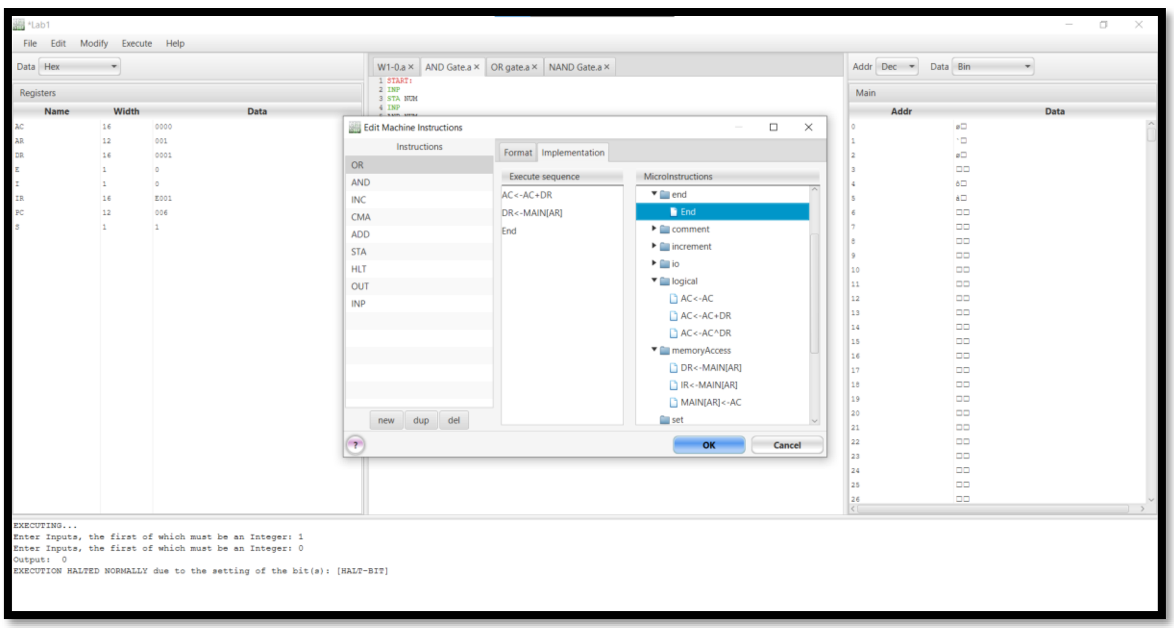


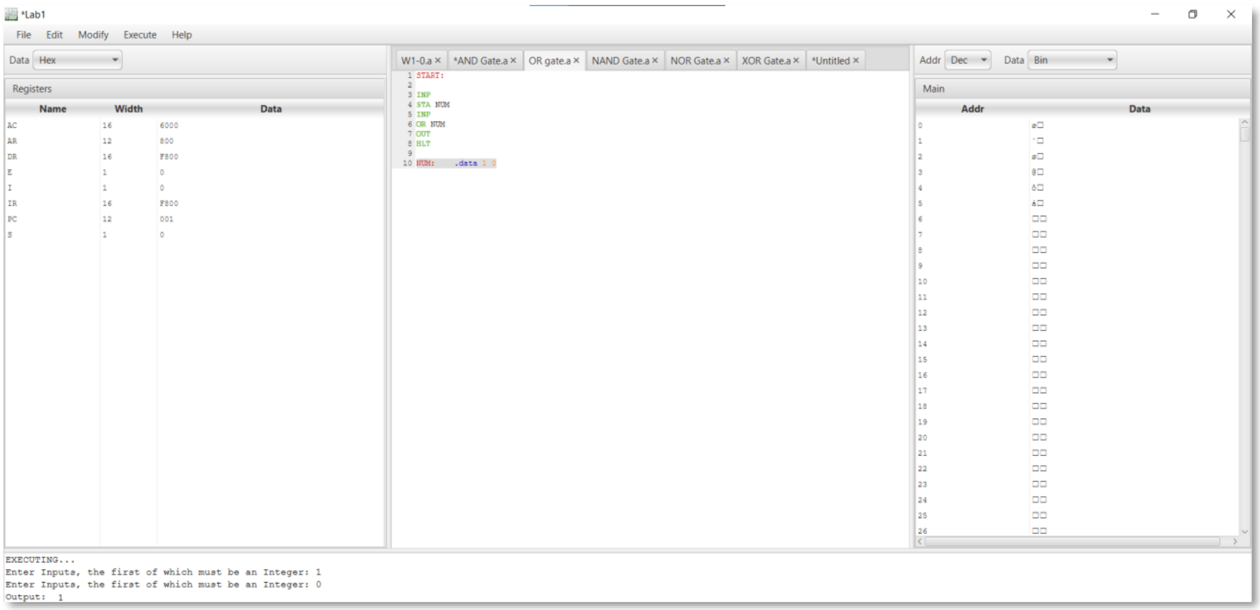
- **This instruction performs a bitwise AND operation between the Accumulator (AC) and a memory location (M).**
- **Set the Opcode for AND as 0.**
- **Set it as a Opp & Address.**

TASK # 2:

OR Operation: Take two inputs, perform the OR operation, and display the result with justification.


Edit Machine Instructions
Instructions
OR
AND
INC
CMA
ADD
STA
HLT
OUT
INP
Format
Implementation
Instruction
Length: 16
Opcode
0x4
All Fields
OP
ADDR
REGISTER
To add fields, drag them in from the list of fields on the right.
To delete fields, drag them out away from the other fields.
Assembly
OK
Cancel
EXECUTING...
Enter Inputs, the first of which must be an Integer: 1
Enter Inputs, the first of which must be an Integer: 0
Output: 0
EXECUTION HALTED NORMALLY due to the setting of the bit(s): [HALT-BIT]

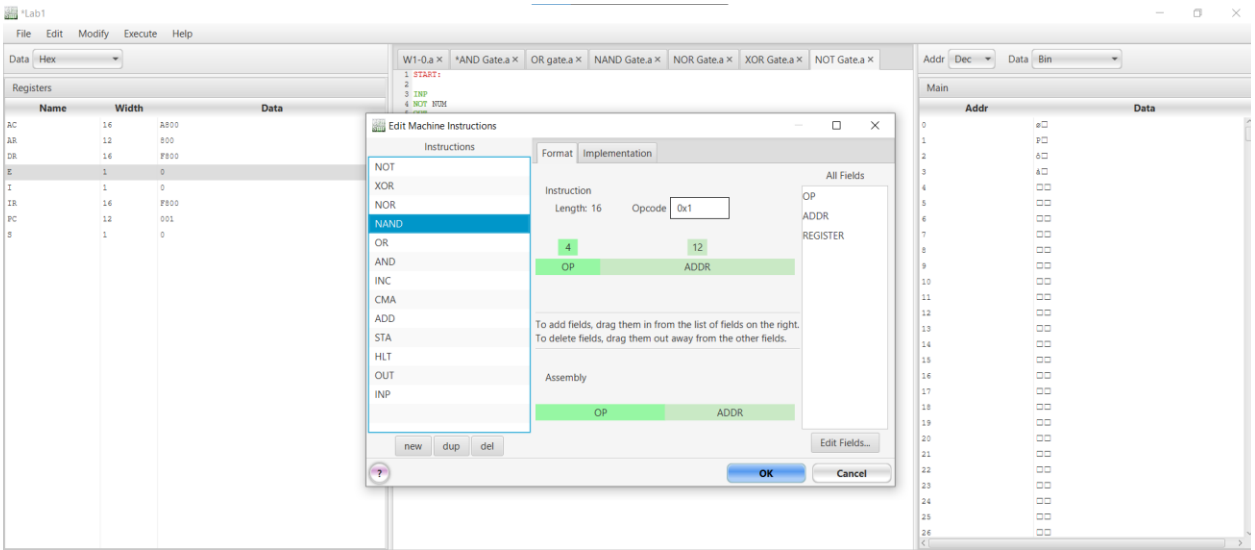

Edit Machine Instructions
Instructions
Format
Implementation
Execute sequence
AC<-AC+DR
DR<-MAIN[AR]
End
MicroInstructions
end
End
comment
increment
io
logical
AC<-AC
AC<-AC+DR
AC<-AC^DR
memoryAccess
DR<-MAIN[AR]
IR<-MAIN[AR]
MAIN[AR]<-AC
set
OK
Cancel
EXECUTING...
Enter Inputs, the first of which must be an Integer: 1
Enter Inputs, the first of which must be an Integer: 0
Output: 0
EXECUTION HALTED NORMALLY due to the setting of the bit(s): [HALT-BIT]

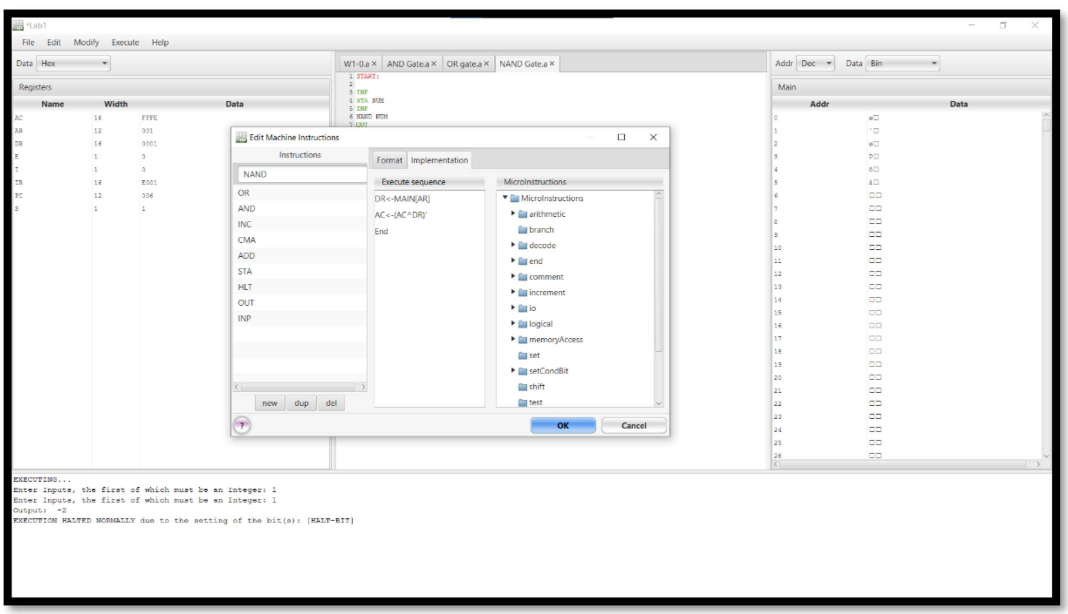


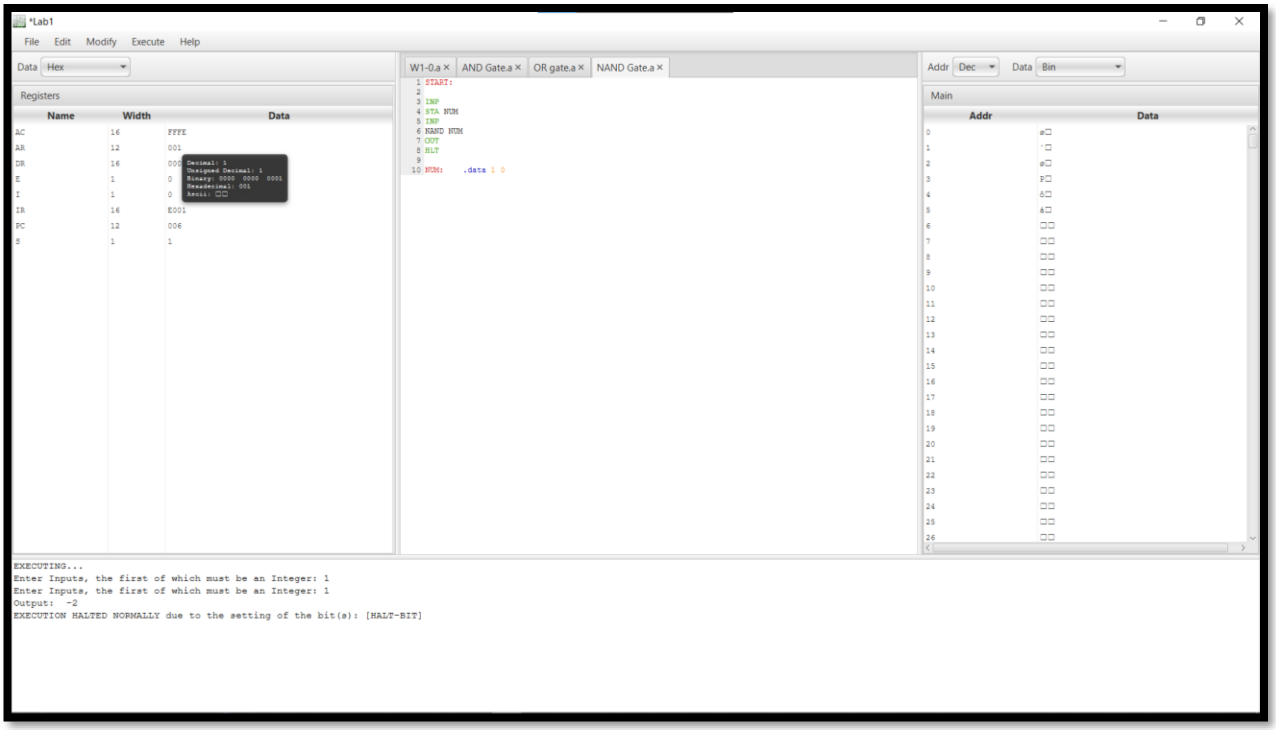
- **This instruction performs a bitwise OR operation between the Accumulator (AC) and a memory location (M).**
- **Set the Opcode for OR as 4.**
- **Set it as a Opp & Address.**

NAND GATE:

NAND Operation: Take two inputs, perform the NAND operation, and display the result with justification.


*Lab1
File Edit Modify Execute Help
Data Hex
Registers
Name Width Data
AC 16 A800
AR 12 800
DR 16 F800
E 1 0
I 1 0
IR 16 F800
PC 12 001
S 1 0
W1-0.a *AND Gate.a OR gate.a NAND Gate.a NOR Gate.a XOR Gate.a NOT Gate.a
Edit Machine Instructions
Instructions
NOT
XOR
NOR
NAND
OR
AND
INC
CMA
ADD
STA
HLT
OUT
INP
new dup del
Format Implementation
Instruction
Length: 16 Opcode 0x1
4 12
OP ADDR
To add fields, drag them in from the list of fields on the right.
To delete fields, drag them out away from the other fields.
Assembly
OP ADDR
All Fields
OP
ADDR
REGISTER
Edit Fields...
OK Cancel
Addr Dec Data Bin
Main
Addr Data

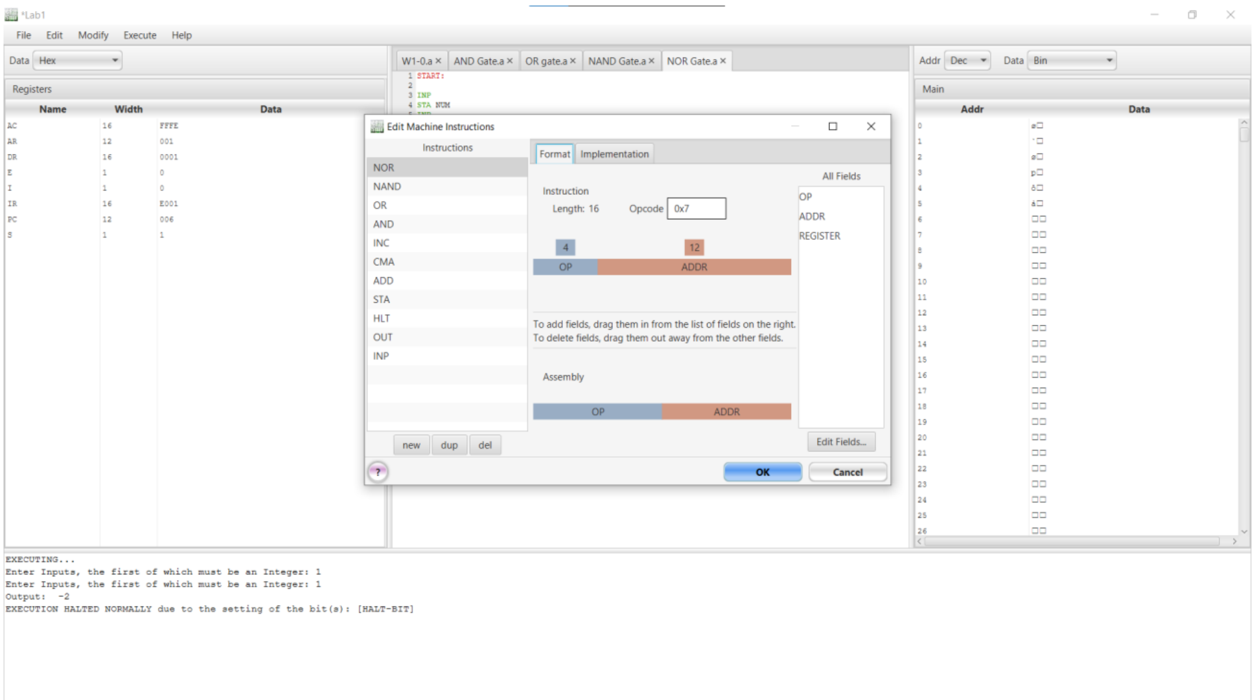

*Lab1
File Edit Modify Execute Help
Registers
Name Width Data
W1-0.a AND Gate.a OR gate.a NAND Gate.a
Edit Machine Instructions
Instructions
NAND
OR
AND
INC
CMA
ADD
STA
HLT
OUT
INP
new dup del
Format Implementation
Execute sequence
DR<-MAIN[AR]
AC<-(AC^DR)'
End
MicroInstructions
arithmetic
branch
decode
end
comment
increment
io
logical
memoryAccess
set
setCondBit
shift
test
OK Cancel
Main
Addr Data
EXECUTING...
Enter inputs, the first of which must be an integer: 1
Enter inputs, the first of which must be an integer: 1
Output: -2
EXECUTION HALTED NORMALLY due to the setting of the bit(s): [HALT-BIT]

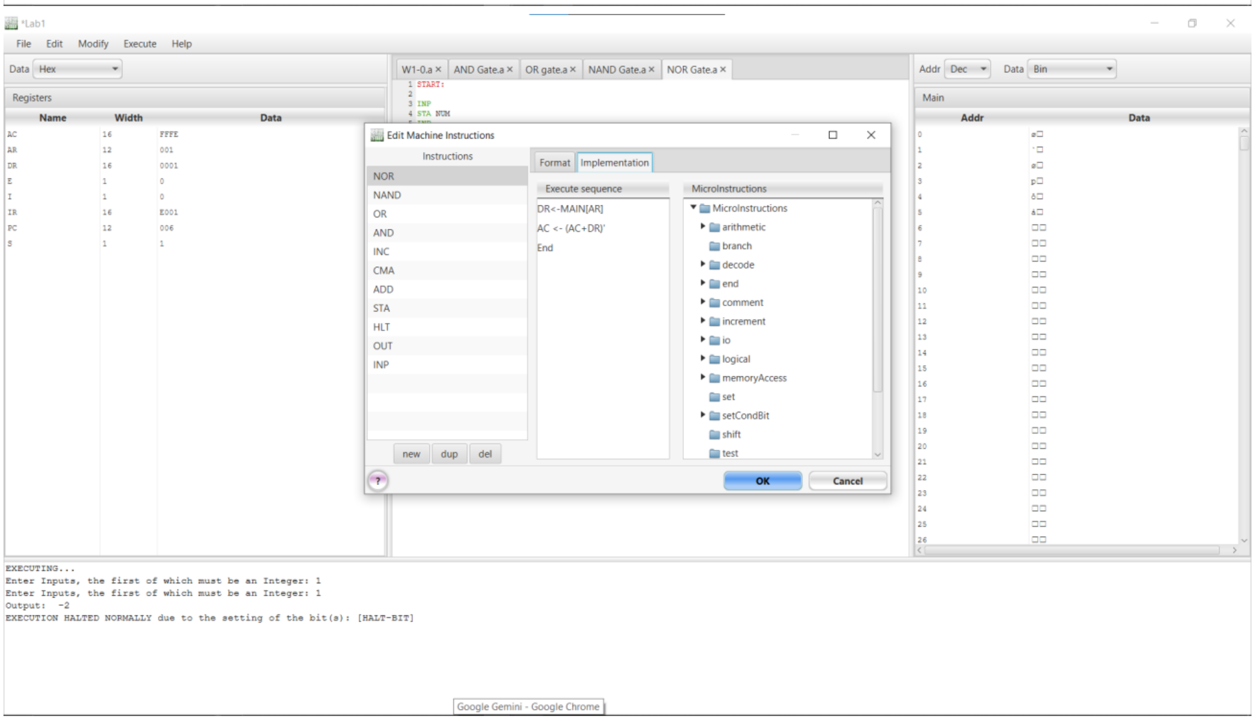


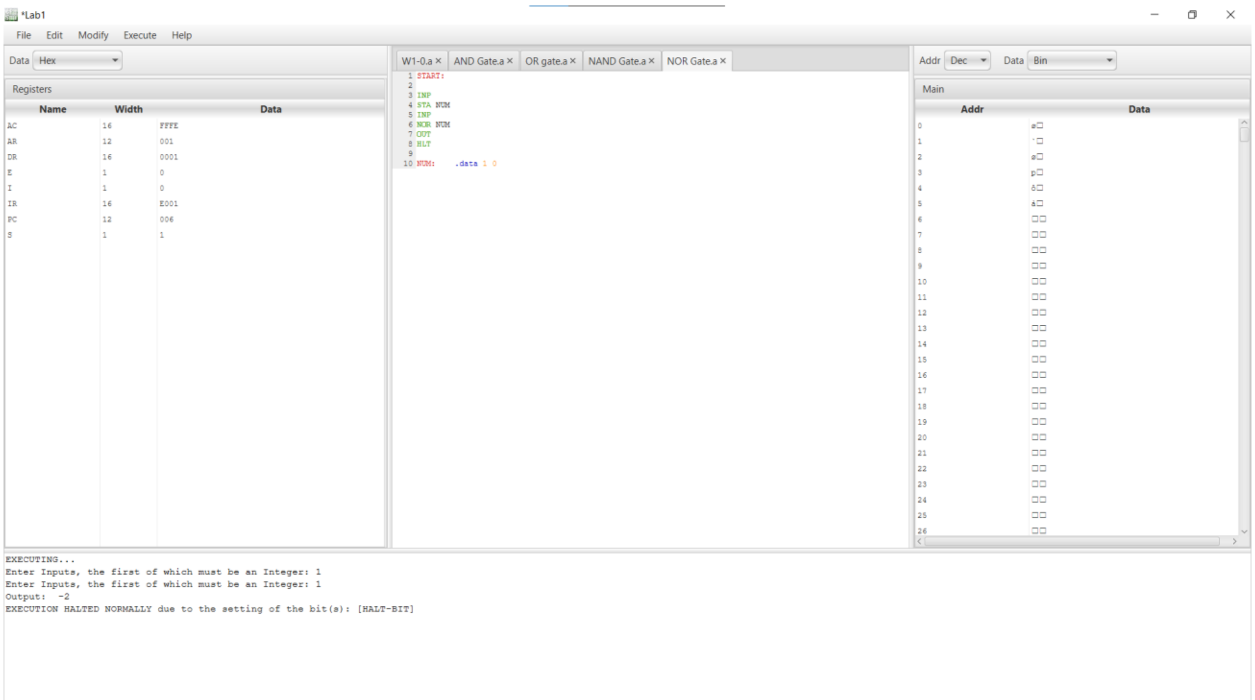
- **This instruction performs a bitwise NAND operation between the Accumulator (AC) and a memory location (M).**
- **Set the Opcode for NAND as 1.**
- **Set it as a Opp & Address.**

Task # 4:

NOR Operation: Take two inputs, perform the NOR operation, and display the result with justification.


*Lab1
File
Edit
Modify
Execute
Help
W1-0.a
AND Gate.a
OR gate.a
NAND Gate.a
NOR Gate.a
Edit Machine Instructions
Instructions
NOR
NAND
OR
AND
INC
CMA
ADD
STA
HLT
OUT
INP
Format
Implementation
Instruction
Length: 16
Opcode
0x7
OP
ADDR
All Fields
REGISTER
To add fields, drag them in from the list of fields on the right.
To delete fields, drag them out away from the other fields.
Assembly
new
dup
del
Edit Fields...
OK
Cancel
EXECUTING...
Enter Inputs, the first of which must be an Integer: 1
Enter Inputs, the first of which must be an Integer: 1
Output: -2
EXECUTION HALTED NORMALLY due to the setting of the bit(s): [HALT-BIT]

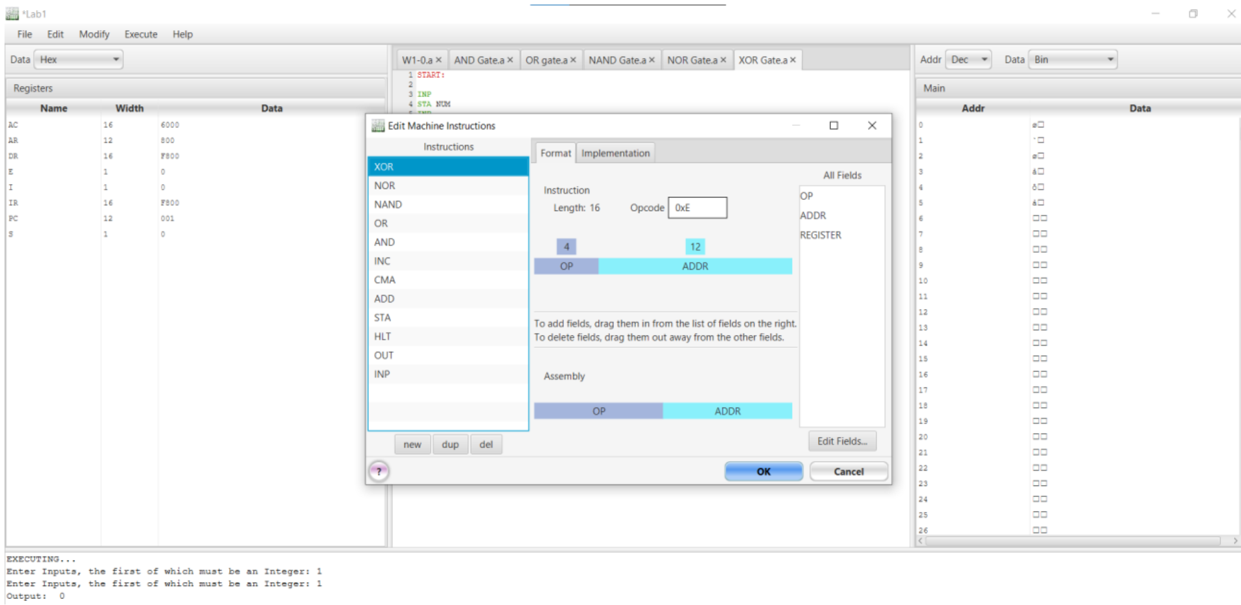

*Lab1
File
Edit
Modify
Execute
Help
Edit Machine Instructions
Format
Implementation
Execute sequence
DR<-MAIN[AR]
AC <- (AC+DR)'
End
MicroInstructions
arithmetic
branch
decode
end
comment
increment
io
logical
memoryAccess
set
setCondBit
shift
test
OK
Cancel
EXECUTING...
Enter Inputs, the first of which must be an Integer: 1
Enter Inputs, the first of which must be an Integer: 1
Output: -2
EXECUTION HALTED NORMALLY due to the setting of the bit(s): [HALT-BIT]
Google Gemini - Google Chrome

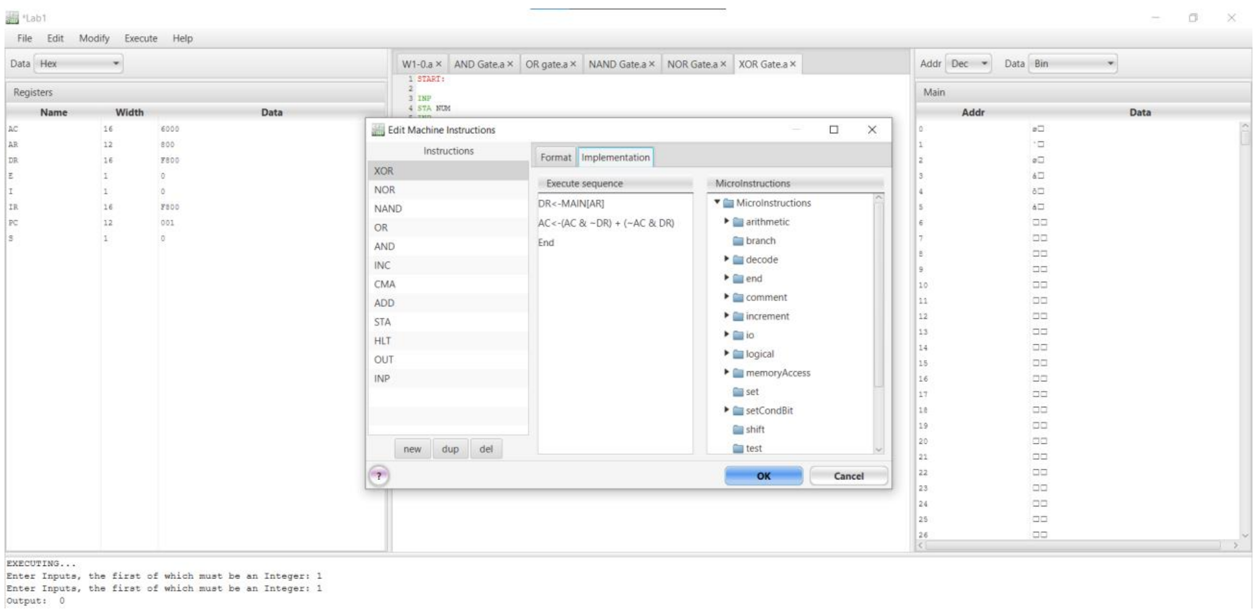


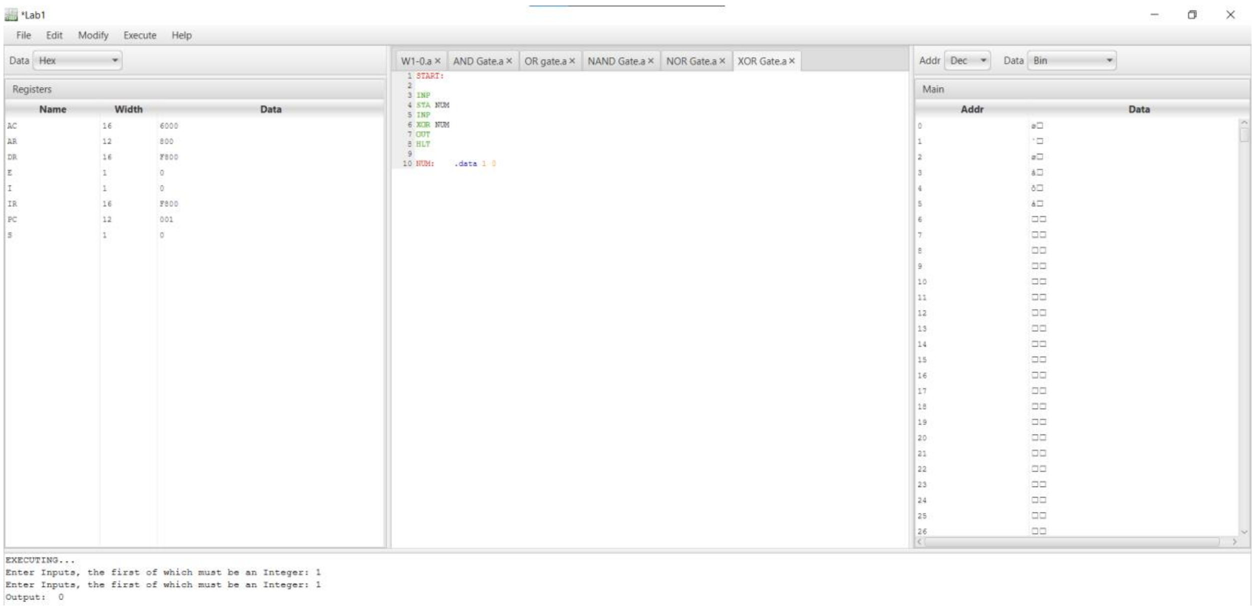
- **This instruction performs a bitwise NOR operation between the Accumulator (AC) and a memory location (M).**
- **Set the Opcode for NOR as 7.**
- **Set it as a Opp & Address.**

TASK # 5:

XOR Operation: Take two inputs, perform the XOR operation, and display the result with justification.


*Lab1
File Edit Modify Execute Help
Data Hex
Registers
Name Width Data
AC 16 6000
AR 12 800
DR 16 F800
E 1 0
I 1 0
IR 16 F800
PC 12 001
S 1 0
W1-0.a AND Gate.a OR gate.a NAND Gate.a NOR Gate.a XOR Gate.a
Edit Machine Instructions
Instructions
XOR
NOR
NAND
OR
AND
INC
CMA
ADD
STA
HLT
OUT
INP
Format Implementation
Instruction
Length: 16 Opcode 0xE
4 12
OP ADDR
All Fields
OP
ADDR
REGISTER
To add fields, drag them in from the list of fields on the right.
To delete fields, drag them out away from the other fields.
Assembly
OP ADDR
new dup del
Edit Fields...
OK Cancel
Addr Dec Data Bin
Main
Addr Data
EXECUTING...
Enter Inputs, the first of which must be an Integer: 1
Enter Inputs, the first of which must be an Integer: 1
Output: 0

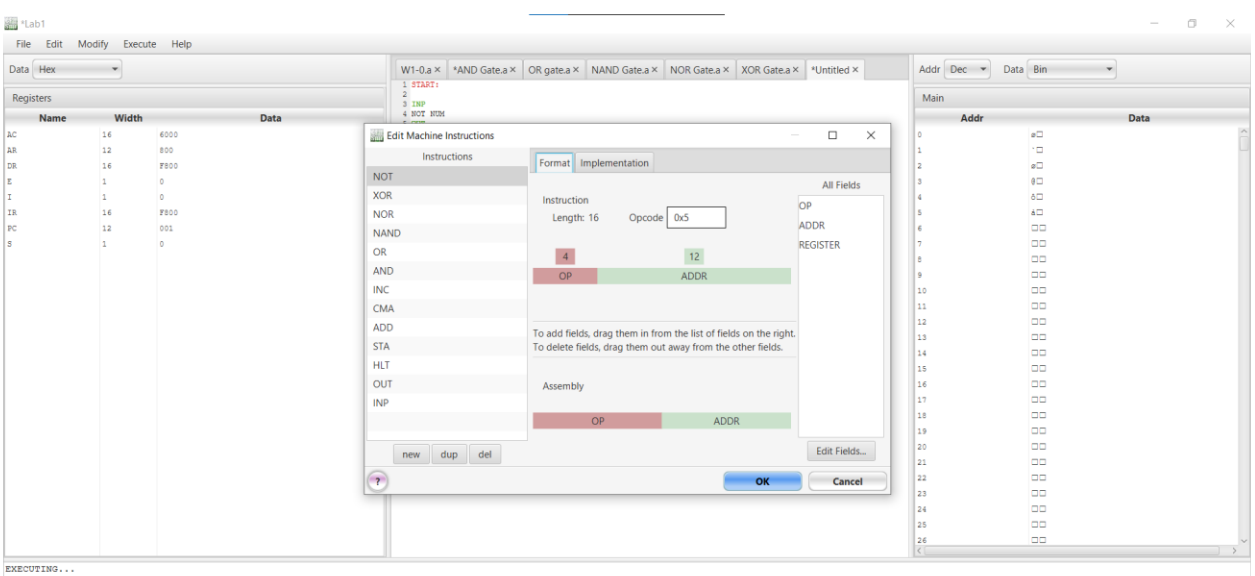

*Lab1
File Edit Modify Execute Help
Data Hex
Registers
Name Width Data
AC 16 6000
AR 12 800
DR 16 F800
E 1 0
I 1 0
IR 16 F800
PC 12 001
S 1 0
W1-0.a AND Gate.a OR gate.a NAND Gate.a NOR Gate.a XOR Gate.a
Edit Machine Instructions
Instructions
XOR
NOR
NAND
OR
AND
INC
CMA
ADD
STA
HLT
OUT
INP
Format Implementation
Execute sequence
DR<-MAIN[AR]
AC<-(AC & ~DR) + (~AC & DR)
End
MicroInstructions
MicroInstructions
arithmetic
branch
decode
end
comment
increment
io
logical
memoryAccess
set
setCondBit
shift
test
new dup del
OK Cancel
Addr Dec Data Bin
Main
Addr Data
EXECUTING...
Enter Inputs, the first of which must be an Integer: 1
Enter Inputs, the first of which must be an Integer: 1
Output: 0

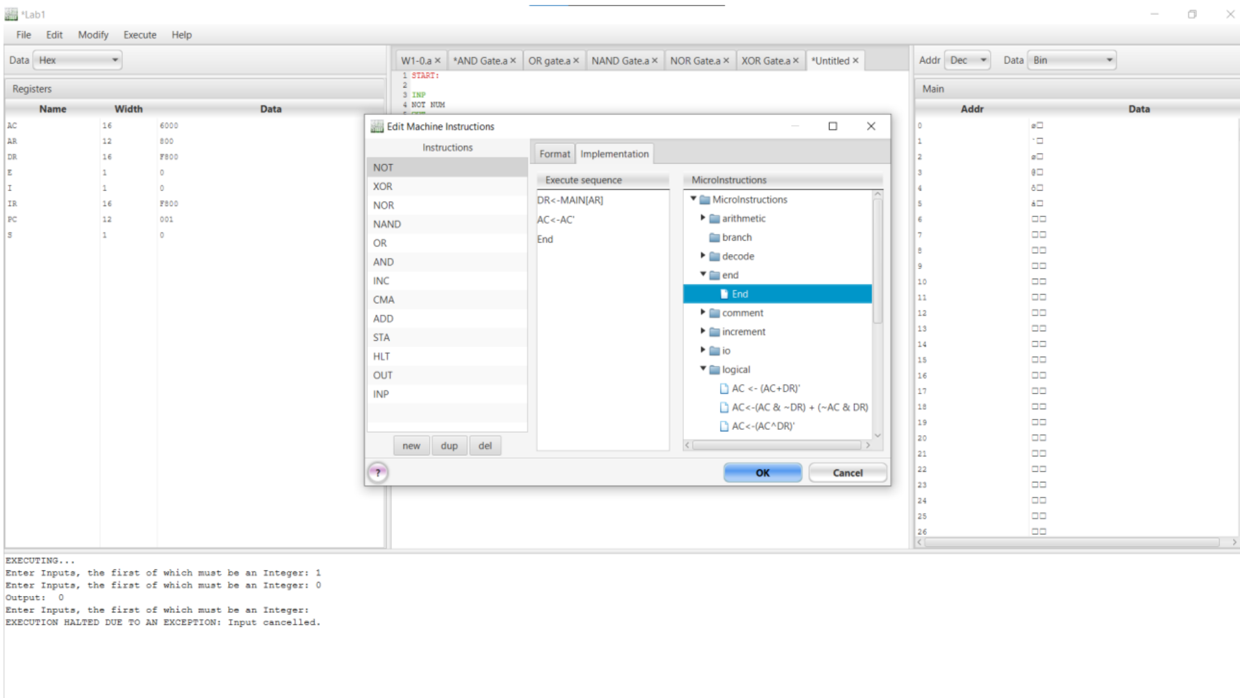


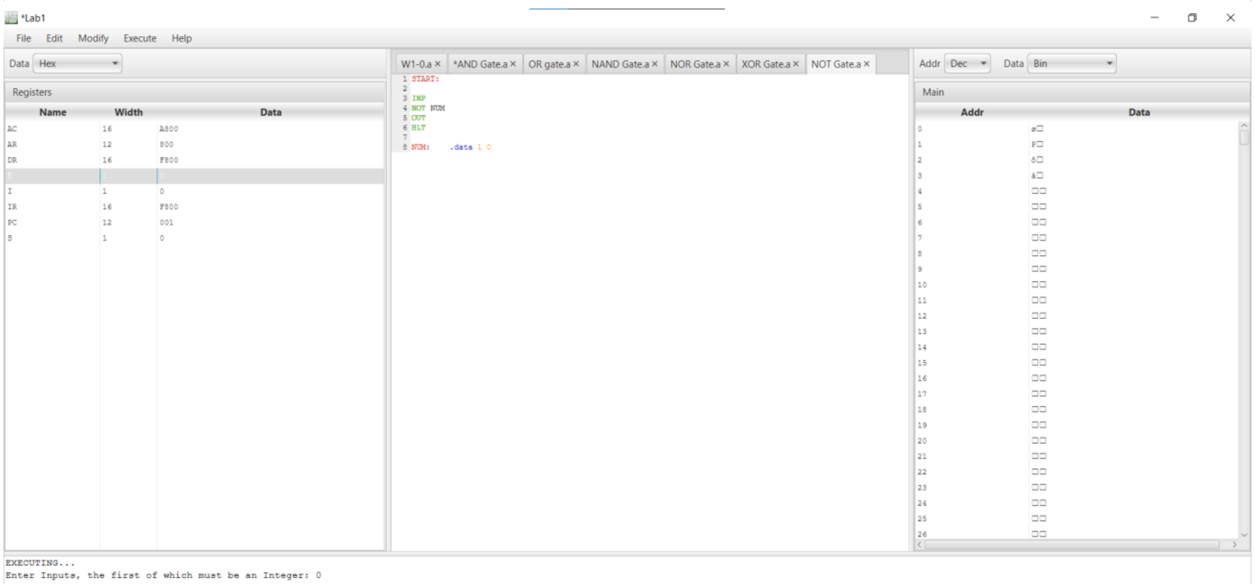
- **This instruction performs a bitwise XOR operation between the Accumulator (AC) and a memory location (M).**
- **Set the Opcode for XOR as E.**
- **Set it as a Opp & Address.**

TASK # 6:

NOT Operation: Take input, perform the NOT operation, and display the result with justification.







- **This instruction performs a bitwise NOT operation between the Accumulator (AC) and a memory location (M).**
- **Set the Opcode for NOT as 5.**
- **Set it as a Opp & Address.**